

Playwright-Notes-3

- **Assertions-** Checks/verifications in our tests-Think of assertions as "checks" after you do something on a page, you want to verify that what you expect to see is actually there. Playwright includes test assertions using the **expect()** method
 - The Simple Idea=>Do something → Then check if the result is what you expected
 - Eg: Is this button visible?
await expect(page.getByRole('button', { name: 'Submit' })).toBeVisible();
 - Write a test → Do actions → Assert that the right things happened → If not, the test fails and tells you why.
 - Assertions are your safety net — they catch when your app behaves unexpectedly.

Note: While using expect() methods import it along with the test component from @playwright/test like so:

```
import {expect, test} from '@playwright/test'
```

- **Types of assertions:**

-Auto-retrying assertions: assertions that keep retrying until the assertion/condition passes or the assertion timeout is reached-asynchronous so it must be used with the **await** keyword

-Non-retrying assertions: assertions that allow you to test any conditions but don't auto-retry, it checks the condition/test once. Most of the time, webpages show info asynchronously and using non-retrying assertions can lead to a flaky test. A flaky test is a test that sometimes passes and sometimes fails — without any changes to the code.

Note: Auto-retrying assertions are more preferred or used

- **Commonly used Auto-retrying assertions:**

1. await expect(locator).toHaveCount()- checks whether specific number of elements are present on the page or not

In VSCode:

```
await page.goto("https://www.saucedemo.com/")  
await expect(page.locator("#login-button")).toHaveCount(1)
```

First we write a test script to navigate to the **saucedemo website** using the **.goto() method**. Next we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether its present one time on the webpage using the **.toHaveCount method(1)** (**.toHaveCount(<number of times you want to check for the presence of an element>)**)

2. await expect(locator).toBeEnabled()- checks if element is enabled or not

In VSCode:

```
await expect(page.locator("#login-button")).toBeEnabled()
```

Here we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether the button is enabled or not (Checks that the element is not disabled — meaning the user can interact with it (click it, type in it, etc.)) using the **.toBeEnabled()** method

3. await expect(locator).toBeDisabled()- checks if element is disabled or not

In VSCode:

```
await expect(page.locator("#login-button")).toBeDisabled()
```

Here we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether the button is disabled or not (Checks that the element is disabled — meaning the user cannot interact with it (click it, type in it, etc.)) using the **.toBeDisabled()** method (opposite of **.toBeEnabled()** method). **In this case for the Login Button the test fails as the Login Button is enabled and not disabled. When a condition fails the test ends/terminates and doesn't go on to execute the next test script in the test block. So its better to comment it out so that the next test script will get executed.**

4. await expect(locator).toBeVisible()- checks if element is visible or not

In VSCode:

```
await expect(page.locator("#login-button")).toBeVisible()
```

Here we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether the button is visible on the

screen to the user or not using the **.toBeVisible()** method. It checks that the element is actually visible to the user on the screen — not hidden, not invisible, not tucked away off screen.

An element is considered visible when it:

Exists in the DOM

Has non-zero dimensions

Is not hidden via CSS

html<!--  toBeVisible() passes -->
<button id="login-button">Login</button>

<!--  toBeVisible() fails — hidden attribute -->
<button id="login-button" hidden>Login</button>

<!--  toBeVisible() fails — CSS display none -->
<button id="login-button" style="display:none">Login</button>

<!--  toBeVisible() fails — CSS visibility hidden -->
<button id="login-button" style="visibility:hidden">Login</button>

<!--  toBeVisible() fails — CSS opacity zero -->
<button id="login-button" style="opacity:0">Login</button>

5. await expect(locator).toBeHidden()- checks if element is hidden or not

In VSCode:

await expect(page.locator("#login-button")).toBeHidden()

Here we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether the button is hidden on the screen to the user or not using the **.toBeHidden()** method. This works opposite to the **.toBeVisible()** method (refer to the **.toBeVisible()** method). **In this case the test fails as the Login Button is visible on the screen When a condition fails the test ends/terminates and doesn't go on to execute the next test script in the test block. So its better to comment it out so that the next test script will get executed.**

6. await expect(page.locator).toHaveText('text')- checks if an element contains a specified text or not

In VSCode:

```
await expect(page.locator("#login-button")).toHaveText("Login")
```

Here we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether the button consists of the text 'Login' using the **.toHaveText()** method which takes a **specified text** as its parameter.

7.await

expect(page.locator).toHaveAttribute('attribute','attributeValue')-checks if an element contains a specified HTML attribute (can be class, id, type, name, value, etc) and attribute value or not

In VSCode:

```
await  
expect(page.locator("#login-button")).toHaveAttribute("name",  
"login-button")
```

Here we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether the button consists the specified **HTML attribute name** with the value as **login-button** using the **.toHaveAttribute method()** which takes in **the name of the HTML attribute** and the **value of the HTML attribute** as its parameters

8. await expect(page.locator).toHaveId('IdValue')- -checks if an element contains a specified HTML id

In VSCode:

```
await expect(page.locator("#login-button")).toHaveId("login-button")
```

Here we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether the button consists the specified **HTML Id value** or not using the **.toHaveId()** method which takes in the **HTML Id value** as its parameter

9. await expect(page.locator).toHaveClass('ClassValue')- -checks if an element contains a specified HTML class

In VSCode:

```
await expect(page.locator("#login-button")).toHaveClass("submit-button  
btn_action")  
})
```

Here we write a test script to locate the **Login Button** element using **CSS locator (we locate it by id)** and we check whether the button consists the specified **HTML Class value** or not using the **.toHaveClass()** method which takes in the **HTML Class value** as its parameter

10. await expect.toHaveURL('URLValue')- checks if the webpage contains a specified URL or not

In VSCode:

```
await expect(page).toHaveURL("https://www.saucedemo.com/")
```

Here we write a test script to check whether a webpage has a specified url which is **https://www.saucedemo.com/** using the **.toHaveURL()** method which takes in a **specified URL** as its parameter

11. await expect.toHaveTitle('TitleValue')- checks if the webpage contains a specified title or not (the title of the webpage can be obtained from the name of the tab in the browser)

In VSCode:

```
await expect(page).toHaveTitle("Swag Labs")
```

Here we write a test script to check whether a webpage has a specified title which is **Swag Labs** using the **.toHaveTitle()** method which takes in a **specified title** as its parameter

- **Commonly used Non-retrying assertions (refer to Playwright docs for more info):**

1. **expect(value).toBe()-** checks whether the value is equal to another value

Eg: `expect(5).toBe(5)` **//this test passes as 5 is equal to 5**

`expect(5).toBe(3)` **//this test fails as 5 is not equal to 3**

2. **expect(value).toBeLessThan()-** checks whether the value is less than another value

Eg: `expect(4).toBeLessThan(5)` **//this test passes as 4<5**

`expect(5).toBeLessThan(4)` **//this test fails as 5>4 and not 4>5**

- **Negative Matchers**- checks whether the opposite of a condition is true by using **.not**- it inverts or flips the assertion
Eg: **await expect(locator).not.toBeVisible()**

In VSCode:

```
await page.goto("https://www.saucedemo.com/")  
await expect(page).not.toHaveTitle("Instagram")
```

Here we write a test script to navigate to the **saucedemo website** and check that the title of the webpage should not have the title as **Instagram** using the **.not.toHaveTitle()** method

- **Custom Error Message**- A custom error message can be specified as a **second argument/parameter** to the **expect()** method
Syntax: **await expect(page, <custom error message>).not.toHaveTitle(<title>)**
Syntax: **expect(page.locator,<custom error message>).toBeDisabled()**

Eg: **await**

```
expect(page, "Page title should be Swag Labs").not.toHaveTitle("Swag Labs")
```

await

```
expect(page.locator("#login-button"),  
"Login button is enabled").toBeDisabled()
```

- **Soft Assertion**- By default failed tests terminate the test. If soft assertions fail, they don't terminate the test, they just mark the test as failed and go on to execute the next test script
Syntax: **await soft.expect(page.locator).toBeHidden**

Eg: **await expect.soft(page.locator("#login-button")).toBeHidden()**

